

Guía de Parcial

Sistemas Distribuidos y Paralelos

Cómo responder cada pregunta del parcial — qué no puede faltar

¿Cómo usar esta guía? Las cajas  verdes muestran lo que *sí o sí* tenés que incluir para no perder puntos. Las cajas  rojas marcan los errores más comunes que bajaron puntos en el parcial. Las  amarillas son tips para diferenciarte. Todo está basado en el resumen de la materia y las correcciones del parcial anterior.

Pregunta 1. ¿Qué es un Sistema Paralelo?

Estructura de respuesta

Un **sistema paralelo** es la combinación entre un **algoritmo paralelo** y la **arquitectura paralela** sobre la que se implementa.

- **Software paralelo (algoritmo paralelo):** varios procesos o hilos que cooperan para la resolución de un problema, con el objetivo de lograr mayor rendimiento que un algoritmo secuencial.
- **Hardware paralelo (arquitectura paralela):** arquitectura con múltiples unidades de procesamiento que permiten el procesamiento paralelo del software paralelo.

*Fórmula clave: **Paralelismo = Software + Hardware***

Sí o sí tenés que decir esto

- Definir qué es un **algoritmo paralelo** (varios procesos/hilos que cooperan).
- Definir qué es una **arquitectura paralela** (múltiples UPs).
- Decir que **un sistema paralelo es la combinación de ambos**.
- Opcionalmente: mencionar cuándo se usan (grandes volúmenes de datos, algoritmos complejos, tiempo real).

Error común — no hagas esto

No alcanza con decir “es un sistema que ejecuta cosas en paralelo” sin definir las dos componentes. El concepto clave que el profe quiere ver es la distinción *software + hardware*. **Si tenes mal este punto, estas desaprobado.**

Pregunta 2. Arquitectura paralela utilizada en la entrega — GPU

Parte A: Arquitectura del cluster (entrega)

Estructura de respuesta

Hay que clasificar la arquitectura del cluster usando el marco teórico de la materia. No alcanza con describir el hardware; hay que usar los conceptos:

1. **Taxonomía de Flynn:** el cluster es **MIMD** — múltiples UPs ejecutan asincrónicamente diferentes instrucciones sobre diferentes flujos de datos. Podría argumentarse también **MPMD** si las UPs trabajan en distintos puntos del programa.
2. **Modelo de comunicación: híbrido** — dentro de cada nodo hay **memoria compartida (UMA)**, y entre nodos hay **memoria distribuida** (cada nodo accede solo a su memoria local, comunicación por pasaje de mensajes).
3. **Espacio de direcciones: distribuido** entre nodos, **compartido** dentro de cada nodo.
4. **Red de interconexión:** la red que conecta los nodos del cluster. La red de interconexión utilizada es una red estática, ya que su topología queda definida al instalar el sistema y no cambia durante la ejecución. Todos los nodos se encuentran conectados mediante una red Ethernet de 1 Gbit/s.
5. **Granularidad: grano medio** — ni muchos cores simples ni pocos ultra-potentes.

Error común — no hagas esto

- **No confundir** el algoritmo implementado (software, problema de n-cuerpos/n-reinas) con la arquitectura utilizada (hardware, el cluster). Son dos cosas distintas.
- **No describir el cluster en lenguaje coloquial** sin usar los conceptos de Flynn, modelo de comunicación, etc. El profe quiere ver que manejás la clasificación teórica.

Parte B: GPU — qué problemas resuelve mejor y su arquitectura

Estructura de respuesta

Clasificar la GPU usando el mismo marco teórico:

1. **Taxonomía de Flynn:** **SIMD** — todos los hilos ejecutan *la misma instrucción* sobre distintos datos. (En CUDA se habla de SIMT, Single Instruction Multiple Threads, que es una extensión de SIMD.)
2. **Modelo de comunicación: memoria compartida** — el costo de comunicación es demasiado alto para memoria distribuida dado el grano fino.
3. **Espacio de direcciones: compartido** (dentro del SM, memoria compartida; globalmente, memoria global de la GPU).

4. **Red de interconexión:** mencionar aunque sea brevemente.
5. **Granularidad: grano fino (manycore)** — muchos cores simples.

Qué problemas ejecuta mejor: problemas **embarrassingly parallel** (vergonzosamente paralelos), con **poca o nula dependencia de datos** y **poca o nula comunicación**. Ejemplo: multiplicación de matrices.

✓ Sí o sí tenés que decir esto

- Mencionar **SIMD** para la GPU (no MIMD).
- Usar la palabra **manycore** y explicar qué significa.
- Decir que los problemas ideales son **embarrassingly parallel** / **vergonzosamente paralelos**.
- Dar un ejemplo concreto: multiplicación de matrices.

💡 Tip para sumar puntos

Si mencionás que en CUDA la arquitectura real se llama **SIMT (Single Instruction, Multiple Threads)** y que es una extensión de SIMD que agrega flexibilidad dentro de los warps, sumás puntos extra.

Pregunta 3. Etapas de diseño de un programa paralelo — Descomposición

Las 4 etapas (Ian Foster)

📋 Estructura de respuesta

Las etapas son, en orden:

1. **Descomposición / Particionado:** se identifica el trabajo que puede hacerse en paralelo y se divide en **tareas**. Se busca definir un gran número de tareas para tener flexibilidad. *Las tareas no son procesos ni hilos; son “cosas a hacer”.*
2. **Comunicación:** se determinan los patrones de comunicación y/o sincronización entre las tareas identificadas. La comunicación es fuente de overhead y debe minimizarse. Puede clasificarse en: global/local, estructurada/no estructurada, estática/dinámica, síncrona/asíncrona.
3. **Aglomeración:** las tareas y comunicaciones se combinan en tareas de mayor tamaño para adaptarlas al hardware disponible. Objetivo: eliminar overhead de comunicación incrementando la granularidad o replicando cómputo cuando sea más barato que comunicar.
4. **Mapeo:** se asigna cada tarea a una unidad de procesamiento. Objetivo: maximizar la utilización de las UPs, asegurar balance de carga y evitar esperas ociosas (idling).

Puede ser **estático** (antes de la ejecución) o **dinámico** (durante la ejecución).

✓ Sí o sí tenés que decir esto

- Nombrar las 4 etapas correctamente: **Descomposición, Comunicación, Aglomeración, Mapeo**.
- Para cada una: decir *qué se hace y cuál es el objetivo*.
- Aclarar que las **tareas no son procesos ni hilos**.
- En **Mapeo**, mencionar que puede ser estático o dinámico.

Estrategias de descomposición

☰ Estructura de respuesta

- 1. Descomposición recursiva:** se usa en problemas divide-y-vencerás. El problema se divide recursivamente en subproblemas iguales e independientes hasta la granularidad deseada. Los resultados se combinan al finalizar. Ej.: búsqueda del mínimo en un vector, Quicksort, Mergesort.
- 2. Descomposición basada en los datos (dominio):** se identifican los datos sobre los que se realizarán cálculos y se los divide entre las tareas. Las tareas suelen ser iguales o similares actuando sobre distintos datos. Tiene cuatro variantes:
 - *Por datos de salida:* si los cálculos de los resultados son independientes, se asigna una tarea por resultado. Ej.: **multiplicación de matrices**.
 - *Por datos de entrada:* cada tarea tiene una partición de los datos de entrada y realiza el cálculo correspondiente. Suele requerir un paso posterior de recuperación. Ej.: búsqueda del MAX en una lista.
 - *Por datos de entrada y salida:* combinación de las anteriores.
 - *Por datos intermedios:* el problema se descompone en etapas que generan dependencias. Ej.: multiplicación de matrices en dos etapas (multiplicar celdas y luego sumar).
- 3. Descomposición exploratoria:** se aplica a problemas que evolucionan durante la ejecución, explorando un espacio de soluciones en forma de árbol (Branch and Bound). Las tareas se generan dinámicamente. Ej.: N-reinas, Puzzle-15. *Suelen generar speedup superlineal*.
- 4. Descomposición especulativa:** se usa cuando las dependencias entre tareas no se conocen a priori. Se asume cierto resultado y se continúa la ejecución; si se acertó, se gana tiempo; si no, se desperdicia cómputo. Ej.: simulación discreta.
- 5. Descomposición híbrida:** combinación de las técnicas anteriores.

✓ Sí o sí tenés que decir esto

Para la multiplicación de matrices:

- La descomposición más natural es **por datos de salida**.
- Justificación: el cálculo de cada elemento de la matriz resultado es **independiente**

de los demás.

- Por lo tanto, es un problema **embarrassingly parallel** (vergonzosamente paralelo): poca o ninguna dependencia de datos, poca o ninguna comunicación.
- Agregar este último punto — que es vergonzosamente paralelo — es lo que cierra la respuesta.

✗ Error común — no hagas esto

No olvidar decir que la multiplicación de matrices es **vergonzosamente paralela**. No alcanza con decir “por datos de salida”; hay que justificar por qué esa es la natural (independencia de cómputo por resultado).

Pregunta 4. Speedup, Speedup Superlineal y Análisis de Escalabilidad

Definición de Speedup

≡ Estructura de respuesta

El **Speedup** es una métrica de rendimiento que representa el **beneficio relativo** alcanzado al ejecutar un programa luego de aplicar una mejora (en nuestro caso, la paralelización).

$$S = \frac{T_a}{T_b} \quad \text{siendo } T_a \text{ el tiempo antes de la mejora y } T_b \text{ el tiempo después.}$$

En el caso particular de comparar secuencial vs. paralelo:

$$S = \frac{T_s}{T_p} \quad \text{donde } T_s \text{ es el tiempo secuencial y } T_p \text{ el tiempo paralelo con } P \text{ UPs.}$$

Si T_s es el *mejor* algoritmo secuencial conocido, se llama **Speedup absoluto**; si no, es **Speedup relativo**.

Casos posibles:

- $S \leq 1$: la versión paralela no mejora (overhead demasiado alto o problema muy chico).
- $S < P$: hay mejora pero no se aprovechan completamente todas las UPs.
- $S = P$: **Speedup lineal o perfecto** — ideal, poco común en la práctica.
- $S > P$: **Speedup superlineal** — ver abajo.

✖ Error común — no hagas esto

No decir “se compara el programa actual con el programa a comparar” — es confuso. La definición correcta es: se compara el tiempo **sin la mejora** con el tiempo **después de la mejora**. En nuestro caso: tiempo secuencial vs. tiempo paralelo.

Speedup Superlineal

📄 Estructura de respuesta

El **Speedup superlineal** ocurre cuando $S > P$. Aunque parezca contradictorio, tiene causas concretas:

1. **El algoritmo secuencial no es el más eficiente:** la versión paralela incluye optimizaciones que también podrían haberse aplicado al secuencial pero no se hicieron (en ese caso el speedup es relativo, no absoluto).
2. **La versión paralela hace menos trabajo:** por ejemplo, en una búsqueda sobre un grafo, un proceso puede encontrar la solución y avisar al resto para que detengan la búsqueda (descomposición exploratoria → speedup superlineal).
3. **Beneficios por la arquitectura:** al usar varios procesadores se dispone de mayor capacidad de caché. Datos que no entraban en la caché de una sola CPU pueden distribuirse entre las cachés de varias UPs, mejorando el rendimiento.

Eficiencia

📄 Estructura de respuesta

La **eficiencia** indica el porcentaje de tiempo en que las UPs están realizando trabajo útil.

$$E = \frac{S}{P} \in (0, 1]$$

Generalmente $E \in (0, 1)$. Un valor cercano a 1 significa que las UPs se usan casi todo el tiempo para trabajo útil.

✔ Sí o sí tenés que decir esto

Definir **eficiencia** antes de usarla en el análisis de escalabilidad. Si la usás sin definirla, es un concepto incompleto en la respuesta.

Leyes de Amdahl y Gustafson

Estructura de respuesta

Ley de Amdahl (tamaño de problema *fijo*):

Existe una fracción f del programa que *no es paralelizable*. Sin importar cuántas UPs se agreguen, el speedup máximo alcanzable es $\frac{1}{f}$. La ley de Amdahl está asociada a la **escalabilidad fuerte**.

¿Cómo verificarla en la tabla? Mirando **por columnas** (tamaño de problema fijo): el speedup debería aumentar hasta un máximo y luego estancarse.

Ley de Gustafson (tamaño de problema *variable*):

A medida que crece el tamaño del problema, la fracción no paralelizable se vuelve proporcionalmente más chica, por lo que es posible alcanzar mayor speedup. Con un tamaño de problema lo suficientemente grande, el speedup puede acercarse a P . La ley de Gustafson está asociada a la **escalabilidad débil**.

¿Cómo verificarla en la tabla? Mirando **por filas** (UPs fijas, tamaño variable): el speedup debería crecer al aumentar el tamaño del problema.

Importante: *ambas leyes siempre se cumplen*. Si el speedup no llega al ideal, no es porque la ley no se cumpla, sino porque hay **overhead** u otros factores. La ley de Gustafson dice que con infinitas UPs se llega al speedup lineal; si no se llega, es porque las UPs son finitas o hay overhead.

Error común — no hagas esto

Error frecuente: decir que “la ley de Gustafson no se cumple” porque el speedup no llega al ideal en la tabla. *Las leyes siempre se cumplen*. Si no se llega al speedup ideal, la causa es el **overhead** o la cantidad limitada de UPs, no una violación de la ley. En el oral puede aceptarse decir que “no se observa claramente el efecto de Gustafson”, pero no que la ley “no se cumple”.

Escalabilidad

Estructura de respuesta

Un programa paralelo puede ser:

- **Fuertemente escalable:** mantiene la eficiencia constante *sin* incrementar el tamaño del problema (solo aumentando las UPs). Se analiza mirando la tabla de eficiencia **por columnas**. Puede que sea fuertemente escalable a partir de cierto valor de UPs.
- **Débilmente escalable:** mantiene la eficiencia constante al incrementar *simultáneamente* el tamaño del problema y el número de UPs. Se analiza mirando la tabla de eficiencia **por la diagonal principal**.
- **No escalable:** no se cumple ninguno de los casos anteriores.

✓ Sí o sí tenés que decir esto

- **Definir** qué es escalabilidad fuerte y débil antes de aplicarlas.
- **Explicitar** qué parte de la tabla se mira para cada tipo (columnas vs. diagonal).
- Si el sistema es fuertemente escalable *a partir de cierto valor*, mencionarlo: “a partir de 32 UPs es fuertemente escalable”.
- Siempre **justificar** la conclusión con los datos de la tabla, no solo enunciarla.
- Definir **eficiencia** como concepto antes de usarla.

💡 Tip para sumar puntos

Si la tabla tiene una diagonal principal con valores de eficiencia aproximadamente constantes, podés concluir escalabilidad débil. Si alguna columna tiene valores constantes, fuerte. Podés concluir *ambas* para distintas regiones de la tabla.

✗ Error común — no hagas esto

No analizar escalabilidad sin antes **definir los conceptos**. Si usás las palabras “fuertemente escalable” o “débilmente escalable” sin explicar qué significan, la respuesta queda incompleta aunque la conclusión sea correcta.

⚡ Checklist rápido antes de entregar

- ✓ P1: Definí *sistema paralelo* como combinación de software + hardware paralelo.
- ✓ P2: Usaste la taxonomía de Flynn, modelo de comunicación, espacio de direcciones y granularidad para clasificar el cluster *y* la GPU por separado.
- ✓ P2: Dijiste MIMD para el cluster y SIMD/manycore para la GPU.
- ✓ P2: Mencionaste “embarrassingly parallel” para la GPU.
- ✓ P3: Nombraste las 4 etapas de Foster con qué hace cada una.
- ✓ P3: Describiste todas las estrategias de descomposición.
- ✓ P3: Dijiste que la multiplicación de matrices usa descomposición por datos de salida *y* que es vergonzosamente paralela.
- ✓ P4: Definiste Speedup con la fórmula correcta.
- ✓ P4: Explicaste las 3 causas del Speedup superlineal.
- ✓ P4: Definiste eficiencia antes de usarla.
- ✓ P4: Explicaste Amdahl (tamaño fijo, columnas) y Gustafson (tamaño variable, filas).
- ✓ P4: Definiste escalabilidad fuerte (columnas) y débil (diagonal) y las aplicaste a la tabla.
- ✓ P4: No dijiste que las leyes de Amdahl/Gustafson “no se cumplen”.